



P01-01 · T1 Repo Scaffold, Toolchain & Banned-Pattern Gate — DEVOPS



T1 exists so that reliability invariant **R1 — no silent failure** is enforced by the build system on day one rather than by discipline on day two hundred. Paste these files, run the two commands at the bottom, and the repo becomes physically hostile to `unwrap()`.

1. Workspace root — `Cargo.toml`

```
[workspace]
resolver = "2"
members = [
    "crates/aura-core",
    "crates/aura-catalog",
    "crates/aura-ingest",
    "crates/aura-jobs",
    "crates/aura-perf",
    "crates/aura-app",
    "crates/aura-cli",
    "xtask",
]

[workspace.package]
version      = "0.1.0"
edition      = "2021"
rust-version = "1.83"
```

```

license      = "UNLICENSED"
authors      = ["AURA"]

[workspace.dependencies]
# Errors and results
thiserror    = "2"
# Serialisation
serde        = { version = "1", features = ["derive"] }
serde_json   = "1"
toml         = "0.8"
# Async runtime: current_thread by default, explicit pools where needed
tokio        = { version = "1", features = ["rt", "rt-multi-thread", "macros", "sync", "time", "fs", "signal"] }
# Data
rusqlite     = { version = "0.32", features = ["bundled", "backup", "functions", "limits"] }
uuid         = { version = "1", features = ["v4", "v7", "serde"] }
blake3       = "1"
time         = { version = "0.3", features = ["formatting", "parsing", "macros", "serde-well-known"] }
# Filesystem
walkdir      = "2"
fs4          = "0.9"
dirs         = "5"
# Imaging metadata (no decode in phase 01)
kamadak-exif = "0.5"
# Observability
tracing      = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter", "json"] }
# Concurrency
rayon        = "1"
crossbeam-channel = "0.5"
parking_lot  = "0.12"

```

```

# Testing
proptest      = "1"
tempfile      = "3"
insta         = { version = "1", features = ["json"] }
criterion     = { version = "0.5", features = ["html_reports"]
}

[profile.release]
opt-level     = 3
lto           = "thin"
codegen-units = 1
panic         = "abort"
strip         = "debuginfo"

# Release-speed builds for the fixture generator and benchmarks
[profile.bench]
inherits      = "release"
debug         = true

# Fast iteration without losing overflow checks
[profile.dev]
overflow-checks = true
debug           = 1

[profile.dev.package."*"]
opt-level = 2

```

Why `panic = "abort"` in release. With R1 enforced, a panic in production means a bug we did not anticipate, and unwinding through FFI boundaries into LibRaw and ONNX Runtime is undefined behaviour waiting to happen. Abort plus the out-of-process crash reporter from [19-OBSERVABILITY-AND-FORENSICS.md](#) produces a usable report; unwinding produces a corrupted catalog.

2. Toolchain pin — `rust-toolchain.toml`

```
[toolchain]
channel      = "1.83.0"
components  = ["rustfmt", "clippy", "rust-src"]
targets      = ["x86_64-pc-windows-msvc", "aarch64-apple-darwin", "x86_64-unknown-linux-gnu"]
profile      = "minimal"
```

A floating toolchain silently changes float codegen and iteration behaviour, which breaks determinism level D0. The version is a contract; bumping it requires re-running the golden determinism suite and an ADR if any digest changes.

3. `rustfmt.toml`

```
edition          = "2021"
max_width        = 100
use_field_init_shorthand = true
newline_style     = "Unix"
reorder_imports   = true
group_imports     = "StdExternalCrate"
```

`newline_style = "Unix"` is not cosmetic: mixed line endings on a Windows checkout change file hashes and break byte-comparison tests.

4. `clippy.toml`

```
# Cognitive load ceilings - a function above these is a review finding
cognitive-complexity-threshold = 20
too-many-arguments-threshold   = 7
type-complexity-threshold      = 300

# Names that must never appear as bare identifiers
disallowed-names = ["foo", "bar", "baz", "tmp", "data", "thing"]
```

```
# Ban the APIs that break determinism or panic silently
[[disallowed-methods]]
path    = "std::option::Option::unwrap"
reason  = "R1: return a coded AuraError instead"

[[disallowed-methods]]
path    = "std::result::Result::unwrap"
reason  = "R1: return a coded AuraError instead"

[[disallowed-methods]]
path    = "std::time::SystemTime::now"
reason  = "R3: wall-clock reads must go through aura_core::clock"

[[disallowed-types]]
path    = "std::collections::HashMap"
reason  = "R3: iteration order is non-deterministic; use BTree
Map or IndexMap"

[[disallowed-types]]
path    = "std::collections::HashSet"
reason  = "R3: iteration order is non-deterministic; use BTree
Set"
```

The `HashMap` ban is the single highest-value lint in this project. Non-deterministic iteration order is the most common cause of "the same wedding produced different output on Tuesday", and it is invisible in code review.

Where a hash map is genuinely needed for performance in a path that does not affect output, use `IndexMap` and add a one-line `// DETERMINISM: read-only lookup, never iterated for output` comment. The reviewer's job is to verify that claim.

5. Lint policy in every crate root

Every `lib.rs` starts with exactly this block:

```

#![forbid(unsafe_code)]
#![deny(
    clippy::unwrap_used,
    clippy::expect_used,
    clippy::panic,
    clippy::todo,
    clippy::unimplemented,
    clippy::indexing_slicing,
    clippy::integer_division,
    clippy::float_cmp,
    clippy::disallowed_methods,
    clippy::disallowed_types,
    clippy::mem_forget,
    clippy::exit,
    missing_debug_implementations,
    unreachable_pub,
    rust_2018_idioms
)]
#![warn(clippy::pedantic, clippy::cargo)]
#![allow(clippy::module_name_repetitions)]

```

`indexing_slicing` is deliberately included. `photos[i]` panics on an empty slice; `photos.get(i)` forces you to decide what an absent photo means, which is exactly the decision that turns a crash into a quarantined item.

The only crates exempt from `forbid(unsafe_code)` are the future `-sys` wrappers in PHASE-02 and PHASE-03, per `16-FFI-AND-GPU-SAFETY.md`. No `-sys` crate exists in phase 01, so the forbid is absolute here.

6. `deny.toml` — supply chain

```

[advisories]
version = 2
yanked  = "deny"
ignore  = []

```

```

[licenses]
version = 2
allow   = ["MIT", "Apache-2.0", "Apache-2.0 WITH LLVM-exception", "BSD-2-Clause", "BSD-3-Clause", "ISC", "Unicode-3.0", "Zlib", "MPL-2.0"]
confidence-threshold = 0.9

[bans]
multiple-versions = "warn"
wildcards          = "deny"
deny = [
  { name = "openssl", reason = "use rustls; no system OpenSSL dependency in a desktop app" },
  { name = "chrono",  reason = "use time; chrono's tz handling has repeatedly broken determinism" },
]

[sources]
unknown-registry = "deny"
unknown-git      = "deny"

```

GPL and AGPL are absent from the allow list on purpose: AURA ships as a proprietary desktop binary, and a copyleft transitive dependency is a licensing failure discovered at the worst possible moment. Note for PHASE-02: **LibRaw is LGPL-2.1 or CDDL dual-licensed**, so it must be linked dynamically and its licence obligations documented in an ADR before that phase starts.

7. `scripts/check-banned.sh`

```

#!/usr/bin/env bash
# R1 enforcement. Runs in stage 1 of CI and in the pre-commit hook.
# Exit 1 on any banned pattern outside permitted locations.
set -euo pipefail

```

```

fail=0

# Files where a panic is acceptable: test code, benches, xtask, and main().
is_exempt() {
    case "$1" in
        */tests/*|*/benches/*|xtask/*|*/main.rs) return 0
    ;;
        *) return 1 ;;
    esac
}

banned_rs='\.unwrap\(\)|\.expect\(|panic!\(|todo!\(|unimplemented!\(|unreachable!\(|process::exit'

while IFS= read -r file; do
    if is_exempt "$file"; then continue; fi
    # Skip lines inside #[cfg(test)] modules is not reliable with grep, so we
    # rely on tests living in tests/ or being gated by the crate-level deny.
    if matches=$(grep -nE "$banned_rs" "$file" | grep -v '// ALLOW-BANNED:'); then
        echo "BANNED PATTERN in $file"
        echo "$matches"
        fail=1
    fi
done < <(find crates -name '*.rs' -type f)

# Determinism patterns
det_banned='HashMap::new|HashSet::new|SystemTime::now|Instant::now|rand::random|thread_rng'
while IFS= read -r file; do
    if is_exempt "$file"; then continue; fi
    if matches=$(grep -nE "$det_banned" "$file" | grep -v '// D

```



```

ETERMINISM:'); then
    echo "DETERMINISM RISK in $file (add a // DETERMINISM: justification if intentional)"
    echo "$matches"
    fail=1
fi
done < <(find crates -name '*.rs' -type f)

# TypeScript: no any, no non-null assertion on IPC boundaries
if [ -d ui/src ]; then
    if matches=$(grep -rnE ':\s*any\b|as any' ui/src --include='*.ts' --include='*.tsx' || true); then
        if [ -n "$matches" ]; then
            echo "BANNED: 'any' in UI source"
            echo "$matches"
            fail=1
        fi
    fi
fi

if [ "$fail" -ne 0 ]; then
    echo ""
    echo "check-banned failed. Do not add // ALLOW-BANNED: to make this pass"
    echo "unless a reviewer has approved it in the PR description."
    exit 1
fi
echo "check-banned: clean"

```

The `// ALLOW-BANNED:` escape hatch exists because a rigid gate gets disabled, and a disabled gate protects nothing. The escape hatch is greppable, and `21-DEFINITION-OF-DONE.md` requires the reviewer to count and justify every occurrence at phase exit. Growth in that count is a review finding.

8. **xtask** — the contract checker

```
// xtask/src/main.rs
// Not shipped. Panics are permitted here.
use std::{collections::BTreeMap, fs, path::Path, process::ExitCode};

fn main() -> ExitCode {
    let args: Vec<String> = std::env::args().skip(1).collect();

    match args.first().map(String::as_str) {
        Some("contracts") => contracts(args.contains(&"--check".to_string())),
        Some("fixtures")  => fixtures(),
        Some("bench")      => bench(&args[1..]),
        _ => { eprintln!("usage: cargo xtask [contracts --check|fixtures|bench <stage>]"); ExitCode::FAILURE }
    }
}

/// Hash every file under a `contract/` directory or listed in
/// contracts.lock,
/// then compare against the recorded digest. Any drift fails CI.
fn contracts(check_only: bool) -> ExitCode {
    let mut current: BTreeMap<String, String> = BTreeMap::new();

    for entry in walkdir::WalkDir::new("crates").into_iter().filter_map(Result::ok) {
        let p = entry.path();
        let is_contract = p.components().any(|c| c.as_os_str() == "contract")
            || p.ends_with("0001_init.sql");
        if is_contract && p.extension().is_some_and(|e| e == "rs" || e == "sql") {
```

```

        let bytes = fs::read(p).expect("read contract file");
        let digest = blake3::hash(&bytes).to_hex().to_string();
        current.insert(p.display().to_string().replace(
            '\\', "/"), digest);
    }
}

for extra in ["migrations/0001_init.sql", "ui/src/ipc/types.ts", "schemas/recipe.v1.json"] {
    if Path::new(extra).exists() {
        let bytes = fs::read(extra).expect("read contract file");
        current.insert(extra.to_string(), blake3::hash(&bytes).to_hex().to_string());
    }
}

let lock_path = Path::new("contracts.lock");
let rendered = current.iter()
    .map(|(k, v)| format!("{v} {k}"))
    .collect::<Vec<_>>()
    .join("\n") + "\n";

if !lock_path.exists() {
    fs::write(lock_path, &rendered).expect("write contracts.lock");
    println!("contracts.lock created with {} entries", current.len());
    return ExitCode::SUCCESS;
}

let recorded = fs::read_to_string(lock_path).expect("read contracts.lock");
if recorded == rendered {
    println!("contracts: {} entries, all locked", current

```

```

t.len());
    return ExitCode::SUCCESS;
}

eprintln!("CONTRACT DRIFT DETECTED");
let old: BTreeMap<_, _> = recorded.lines()
    .filter_map(|l| l.split_once(" "))
    .map(|(d, p)| (p.to_string(), d.to_string()))
    .collect();
for (path, digest) in &current {
    match old.get(path) {
        None => eprintln!("  NEW      {path}"),
        Some(prev) if prev != digest => eprintln!("  CHAN
GED  {path}"),
        Some(_) => {}
    }
}
for path in old.keys() {
    if !current.contains_key(path) { eprintln!("  REMOVED
{path}"); }
}
eprintln!("\nA frozen contract changed. Write an ADR in d
ocs/adr/ with a grep");
eprintln!("blast radius, get it approved, then re-run wit
hout --check to relock.");
if check_only { ExitCode::FAILURE } else {
    fs::write(lock_path, &rendered).expect("relock");
    eprintln!("contracts.lock updated.");
    ExitCode::SUCCESS
}
}

fn fixtures() -> ExitCode { /* T16, see P01-07 */ ExitCode::S
UCCESS }

```

```
fn bench(_stages: &[String]) -> ExitCode { /* T12, see P01-08
*/ ExitCode::SUCCESS }
```

Why hash-based rather than an honour system. An agent asked "did you change any contracts?" will answer no in good faith after changing three. A digest cannot be mistaken. The ADR requirement then has teeth, because the only path forward is to write one.

9. `.gitignore`

```
/target
/ui/node_modules
/ui/dist
/tests/fixtures/generated
/perf/results
*.sqlite
*.sqlite-wal
*.sqlite-shm
.DS_Store
.env
*.aura-license
```

Note what is **not** ignored: `contracts.lock`, `perf/budgets.toml`, `errors.toml` and `tests/golden/manifest.toml` are all committed. They are the memory of the project.

10. Pre-commit hook — `.github/hooks/pre-commit`

```
#!/usr/bin/env bash
set -euo pipefail
cargo fmt --all -- --check
./scripts/check-banned.sh
cargo clippy --workspace --all-targets -- -D warnings
cargo xtask contracts --check
```

Install with `git config core.hooksPath .github/hooks`. This is the same set as CI stage 1, so a red CI on formatting should never happen.

11. Crate manifests

Each crate keeps its own dependency list narrow, which is what stops `aura-core` from accidentally depending on SQLite and becoming untestable in isolation.

```
# crates/aura-core/Cargo.toml
[package]
name = "aura-core"
version.workspace      = true
edition.workspace      = true
rust-version.workspace = true
```

```
[dependencies]
thiserror.workspace = true
serde.workspace     = true
serde_json.workspace = true
toml.workspace      = true
uuid.workspace      = true
time.workspace      = true
tracing.workspace   = true
dirs.workspace       = true
```

```
[dev-dependencies]
proptest.workspace = true
insta.workspace    = true
tempfile.workspace = true
```

```
# crates/aura-catalog/Cargo.toml
[package]
name = "aura-catalog"
version.workspace = true
edition.workspace = true

[dependencies]
aura-core = { path = "../aura-core" }
```

```
rusqlite.workspace    = true
serde.workspace       = true
serde_json.workspace  = true
time.workspace        = true
tracing.workspace     = true
parking_lot.workspace = true
crossbeam-channel.workspace = true
fs4.workspace         = true
blake3.workspace      = true
```

```
[dev-dependencies]
tempfile.workspace = true
proptest.workspace = true
```

`aura-core` must never depend on `aura-catalog`, `aura-ingest` or `aura-app`. That single rule keeps the error taxonomy usable everywhere without a dependency cycle, and it is worth asserting in a test:

```
// crates/aura-core/tests/no_upward_deps.rs
#[test]
fn core_has_no_workspace_dependencies() {
    let manifest = std::fs::read_to_string("Cargo.toml").expect("manifest");
    for forbidden in ["aura-catalog", "aura-ingest", "aura-jobs", "aura-app"] {
        assert!(!manifest.contains(forbidden), "aura-core must not depend on {forbidden}");
    }
}
```

12. Acceptance criteria for T1

- ☐ `cargo build --workspace --all-targets` succeeds on Windows, macOS and Linux
- ☐ `cargo clippy --workspace --all-targets -- -D warnings` is clean

- ☐ `cargo fmt --all -- --check` is clean
- ☐ `./scripts/check-banned.sh` prints `clean` and exits 0
- ☐ Adding a deliberate `let x: u32 = None.unwrap();` to `aura-core/src/lib.rs` makes **both** Clippy and `check-banned.sh` fail; both outputs pasted, then the line removed
- ☐ Adding a `HashMap::new()` triggers the determinism check; adding `// DETERMINISM: justification` silences it
- ☐ `cargo xtask contracts` creates `contracts.lock`; `--check` then passes; editing one byte of a contract file makes `--check` fail with `CHANGED`, output pasted
- ☐ `cargo deny check` passes, or every exception is recorded with a reason
- ☐ `git config core.hooksPath .githubhooks` documented in the README
- ☐ No crate has a dependency edge that points upward; the `no_upward_deps` test passes

13. Claude Code prompt for T1

Act as `DEVOPS` (Release Engineering). Implement T1 of PHASE-01 exactly as specified on this page: workspace manifest, pinned toolchain, `rustfmt.toml`, `clippy.toml`, the crate-root lint block in all seven crates, `deny.toml`, `scripts/check-banned.sh`, the `xtask contracts` digest checker, `.gitignore`, and the pre-commit hook. Create all seven crates with empty `lib.rs` files carrying the lint block.

Constraints: do not add a dependency that is not in the workspace list; do not weaken any lint to make the build pass; `aura-core` must not depend on any other workspace crate.

Then prove the gates work, in this order, pasting real output for each: (1) build, Clippy, fmt clean; (2) insert `None.unwrap()`, show Clippy fail and `check-banned.sh` fail, remove it; (3) insert `HashMap::new()`, show the determinism check fail, add the justification comment, show it pass, then remove both; (4) `cargo xtask contracts` then `--check` passing, then flip one byte in a contract file and show `CHANGED`, then restore. Do not proceed to T2.